02285 AI and MAS, F26
Partial-order planning and hierarchical task networks

Curriculum for 4:

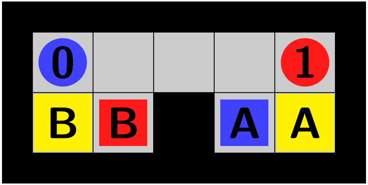
- Section 11.3 *Partial-Order Planning* of **2nd edition** of Russell & Norvig (available on DTU Learn)
- Section 11.4 *Hierarchical Planning* of R&N, 4ed
- Section 11.7 *Analysis of Planning Approaches* of R&N, 4ed

Today's subjects:

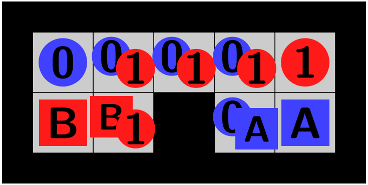
- Partial-order planning.
- Hierarchical task networks.

Divide-and-conquer strategies

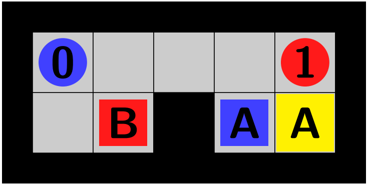
Some common divide-and-conquer strategies, either for computing full solutions to the original problem, or only for more efficiently estimating its cost (compute a heuristic).



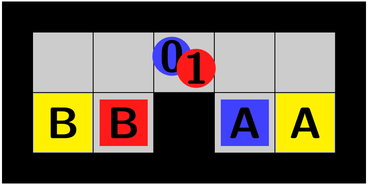
Original level



Relaxation: delete relaxation



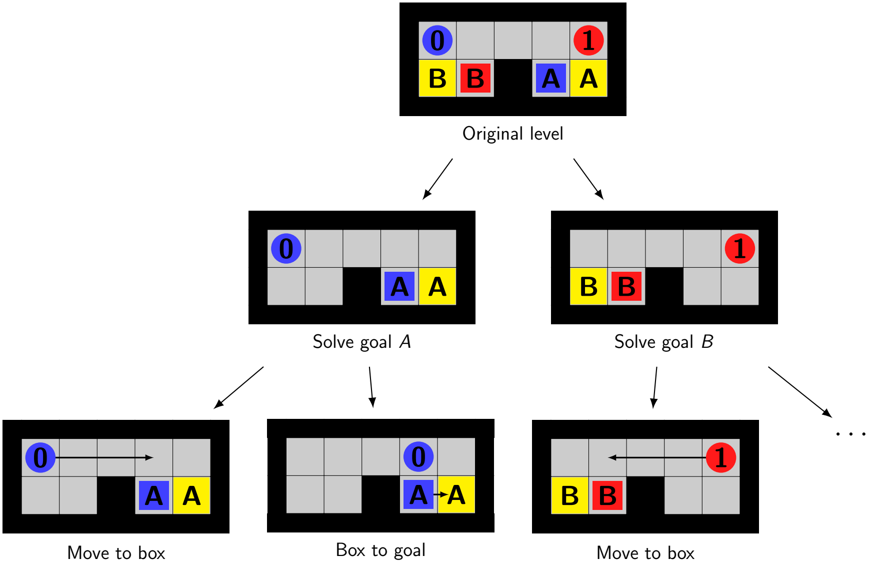
Subgoal serialisation



Subgoal independence

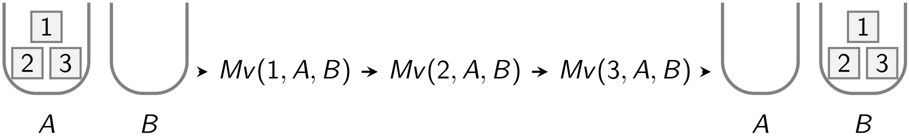
Divide-and-conquer strategies

Problem decomposition can also be hierarchical:



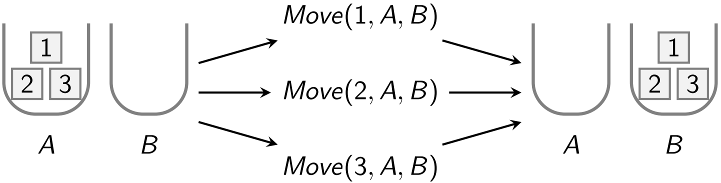
Linear sequences versus partial orders

Standard planning techniques explore **linear sequences of actions**.



This is often not advantageous. E.g. considering in which order to move the boxes will just make the planning process more time-consuming.

We'd like some degree of **partiality** in the ordering of actions, e.g. the order in which to move the boxes or the order in which to buy milk and bananas.



Partial-order planning

A generalised treatment of partiality is found in **partial-order planning (POP)**.

Partial-order planning (POP): A planning technique producing partially ordered plans, ignoring the ordering of independent actions.

Partial-order planning is in particular effective for **partly decomposable** problems like e.g. the gripper problem (or planning a party).

Example: Socks-and-shoes

Putting on your socks and shoes...

Action schemas:

ACTION: *LeftSock*
PRECONDITION:
EFFECT: *LeftSockOn*

ACTION: *LeftShoe*
PRECONDITION: *LeftSockOn*
EFFECT: *LeftShoeOn*

ACTION: *RightSock*
PRECONDITION:
EFFECT: *RightSockOn*

ACTION: *RightShoe*
PRECONDITION: *RightSockOn*
EFFECT: *RightShoeOn*

Initial state: empty.

Goal: $\mathit{LeftShoeOn} \wedge \mathit{RightShoeOn}$.

Note the independence of the actions on the left from the actions on the right. Partial-order planning takes advantage of this.

Partial orders

In partial-order planning a partial order (order induced by directed, acyclic graph) $\prec$ on the required actions is built up in a stepwise fashion.

Example. In the case of the socks-and-shoes problem, the partial order would become:

$$\mathit{LeftSock} \prec \mathit{LeftShoe}, \mathit{RightSock} \prec \mathit{RightShoe}.$$

Expresses: put on left sock **before** left shoe, and put on right sock **before** right shoe.

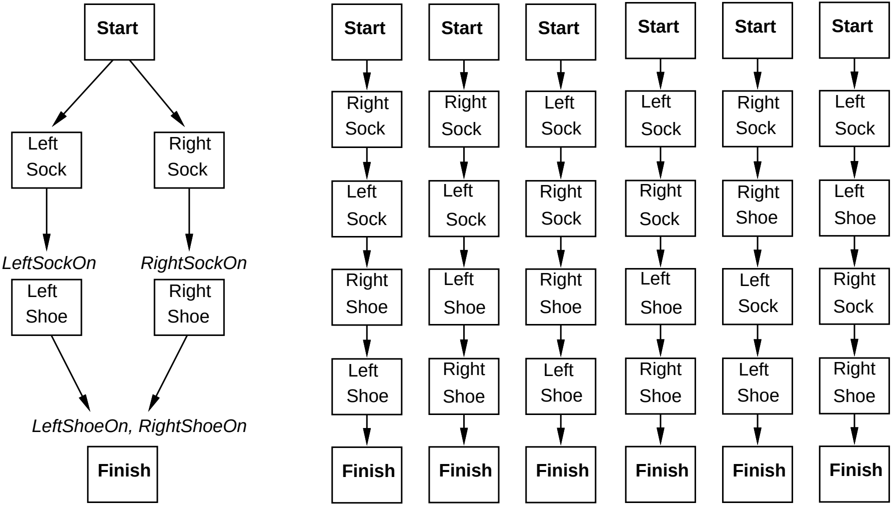
Linearisation of a partial order: Any total order that extends it. Polynomial-time algorithms for linearisation: topological sort.

Definition (Partial orders as solutions)

A partial order $\prec$ is called a **solution** to a planning problem if any **linearisation** of it constitutes a **plan** (action sequence leading from start to goal).

Partial-order planning for socks-and-shoes

Partial Order Plan: **Total Order Plans:**



Compare with the simplified Gripper problem.

Ordering constraints

Definition (Ordering constraint)

An **ordering constraint** is an expression of the form $A \prec B$, where A and B are actions.
Intended interpretation: Action A has to come before action B.

Definition (Labelled ordering constraint, called 'causal link' in R&N)

A **labelled ordering constraint** is an expression of the form $A \overset{c}{\prec} B$, where:

- A and B are actions.
- c is an **effect** of A (one of the literals in the effect of A).
- c is a **precondition** of B (one of the literals in the precondition of B).

$A \overset{c}{\prec} B$ is read: "action A **achieves** (pre)condition c for performing action B" or simply "A **achieves** c for B." E.g.

$$\mathit{RightSock} \overset{\mathit{RightSockOn}}{\prec} \mathit{RightShoe}.$$

Partially ordered plans

Definition (Partially ordered plan)

A **partially ordered plan** is a directed graph where

- **Nodes** represent **actions**.
- **Edges** represent **(labelled) ordering constraints** between actions.

Ordering constraints:

$$\mathit{RightSock} \overset{\mathit{RightSockOn}}{\prec} \mathit{RightShoe}.$$

become edges:

$$\boxed{\mathit{RightSock}} \xrightarrow{\mathit{RightSockOn}} \boxed{\mathit{RightShoe}}$$

POP algorithm (partial-order planning algorithm): Start with the empty partially ordered plan, then iteratively add more actions (nodes) and ordering constraints (edges).

Open preconditions

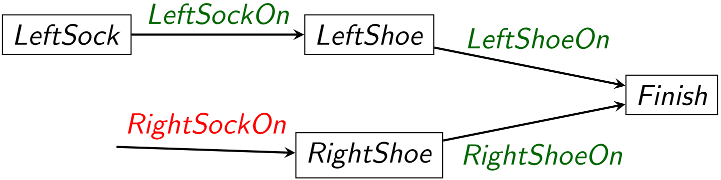
Recall: $A \overset{c}{\prec} B$ reads "A **achieves** c for B."

Let B denote an action occurring in a partially ordered plan, and let c denote one of its preconditions (precondition literals).

Definition (Open precondition)

The precondition c is called **open** if it is not achieved by any action in the plan, that is, the plan doesn't contain any ordering constraint of the form $A \overset{c}{\prec} B$.

Example. In the partially ordered plan below, *RightSockOn* is open, but not *LeftSockOn*.



Conflicts

Definition (Conflict)

An action $D \neq B$ is said to **conflict** with an ordering constraint $A \stackrel{c}{\prec} B$ if D's effect is inconsistent with c (D destroys c). This means that we cannot allow D to be executed between A and B.

Definition (Conflict resolution)

If D conflicts with an ordering constraint $A \stackrel{c}{\prec} B$ we can **resolve** the conflict by adding either the constraint $B \prec D$ or $D \prec A$ (while ensuring the ordering $\prec$ to still be a strict partial order).

Example. The action $Go(N,H)$ conflicts with the ordering constraint $Go(H,N) \stackrel{At(N)}{\prec} Buy(M,N)$: we are not allowed to perform the action $Go(N,H)$ between the actions $Go(H,N)$ and $Buy(M,N)$.

POP algorithm

POP algorithm: Start with the empty plan. Then iteratively do the following:

- Pick an **open precondition** and choose an action that achieves it.
- Resolve any conflicts introduced.

Solution: A partially ordered plan with no open preconditions and no unresolved conflicts will constitute a **solution** to the planning problem (because any linearisation of the ordering $\prec$ of the partially ordered plan will constitute a plan).

Now for the details...

POP algorithm

Initialisation: One starts with the **empty plan** only consisting of the actions *Start* and *Finish*.

Start: An action having the planning problem's start state description as effect.

Finish: An action having the planning problem's goal state description as precondition.

After initialisation the following step is repeated:

Step: Pick an open precondition c on some action B in the current plan and choose an action A that achieves c (either an existing action in the plan or a new one). Then:

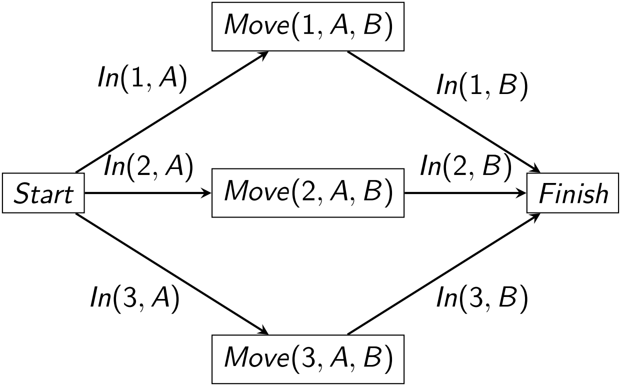
1. Add ordering constraints $A \overset{c}{\prec} B, \mathit{Start} \prec A, A \prec \mathit{Finish}$ (not necessary to draw edges for the two latter).
2. Resolve any conflicts introduced.

Backtrack if:

- An open precondition can't be achieved.
- Some conflict can't be resolved.

POP example: simplified Gripper problem

The following is the partially ordered plan resulting from running the POP algorithm on the simplified Gripper problem with $n = 3$:



Note that the graph constructed only has size $O(n)$, even less than the $O(n^2)$ for greedy-best first GRAPH-SEARCH with an optimal heuristics.

POP example: Milk and bananas

Action schemas:

ACTION: $Buy(x, y)$
PRECONDITION: $Buyable(x) \wedge Place(y) \wedge At(y) \wedge Sells(y, x)$
EFFECT: $Have(x)$

ACTION: $Go(x, y)$
PRECONDITION: $Place(x) \wedge Place(y) \wedge At(x) \wedge Close(x, y)$
EFFECT: $At(y) \wedge \neg At(x)$

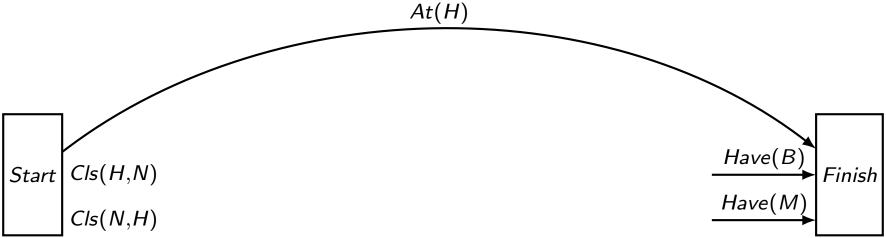
Initial state: $Buyable(Milk) \wedge Buyable(Bananas) \wedge Place(Home) \wedge Place(Netto) \wedge Place(Lidl) \wedge At(Home) \wedge Sells(Netto, Milk) \wedge Sells(Netto, Bananas) \wedge Sells(Lidl, Milk) \wedge Sells(Lidl, Bananas) \wedge Close(Home, Netto) \wedge Close(Netto, Home)$

Goal state: $At(Home) \wedge Have(Milk) \wedge Have(Bananas)$

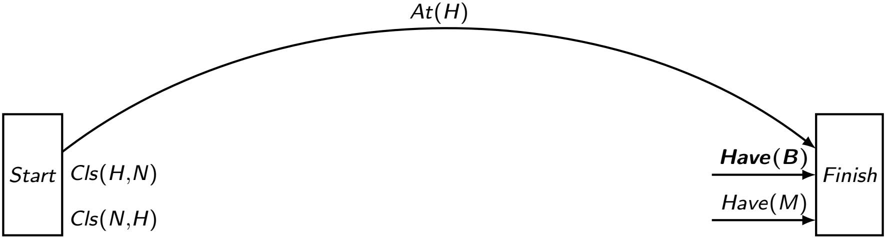
POP example: Milk and bananas

POP example: Milk and bananas

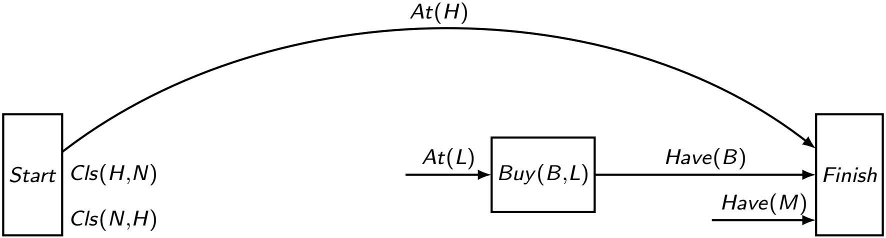
POP example: Milk and bananas



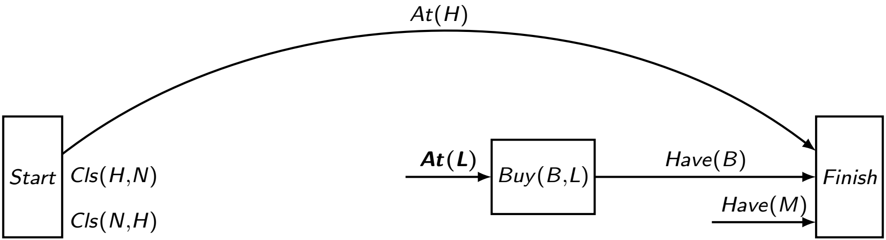
POP example: Milk and bananas



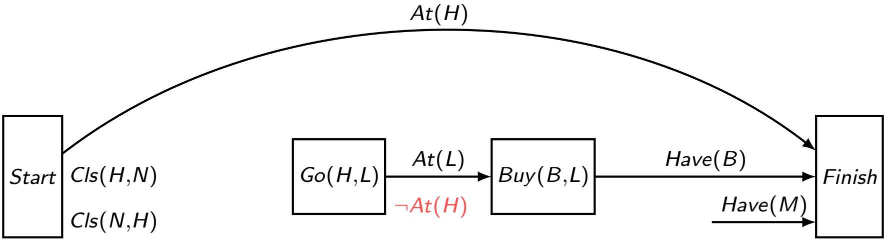
POP example: Milk and bananas



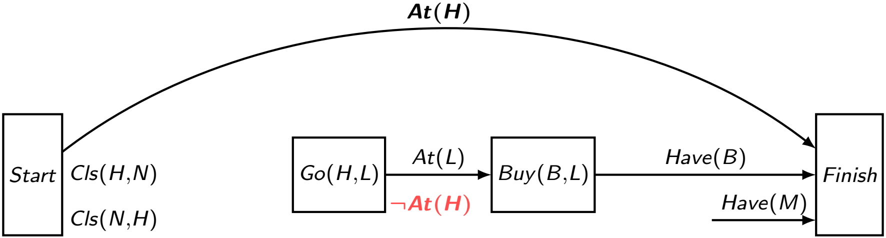
POP example: Milk and bananas



POP example: Milk and bananas



POP example: Milk and bananas

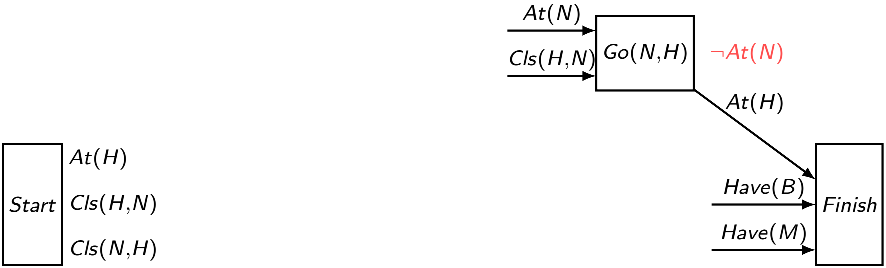


Action $Go(H, L)$ conflicts with the ordering constraint $Start \overset{At(H)}{\prec} Finish$. We can't put $Go(H, L)$ before $Start$ or after $Finish$, so we have to backtrack.

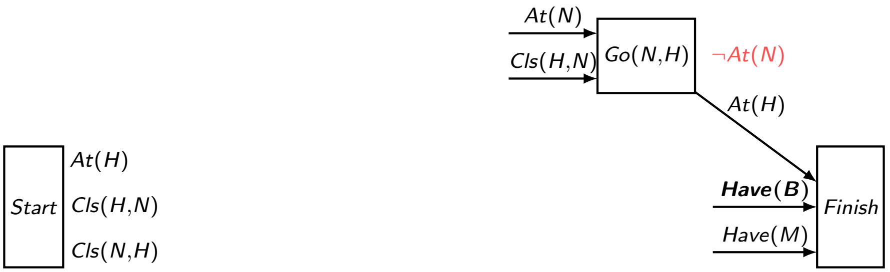
POP example: Milk and bananas

POP example: Milk and bananas

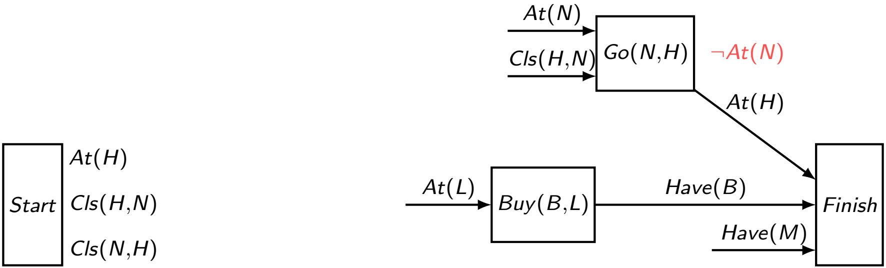
POP example: Milk and bananas



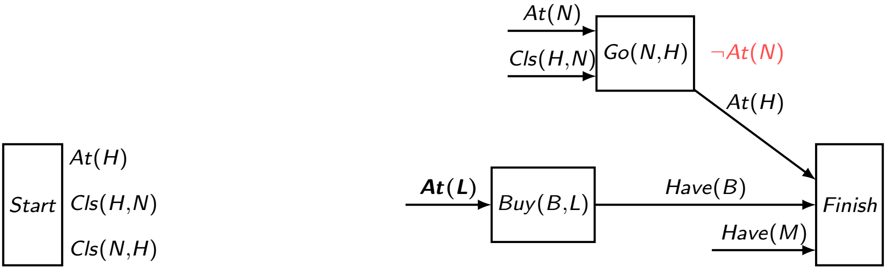
POP example: Milk and bananas



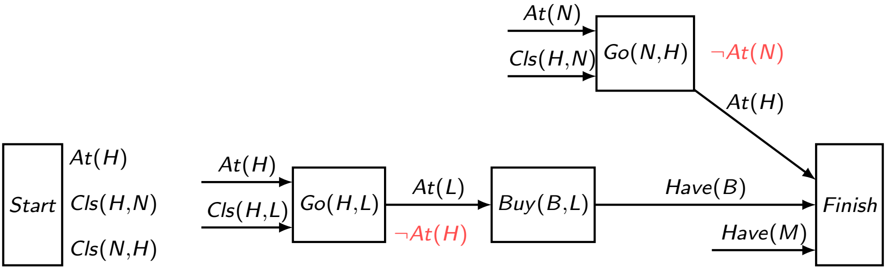
POP example: Milk and bananas



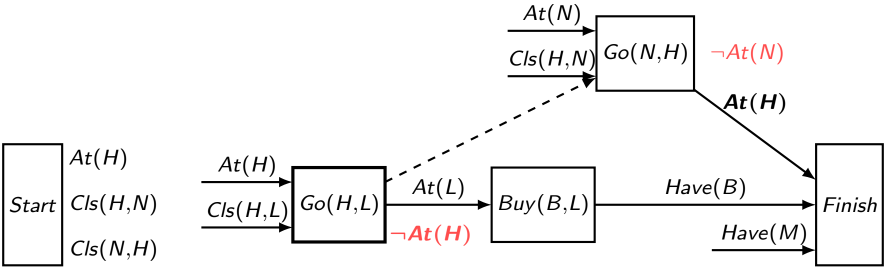
POP example: Milk and bananas



POP example: Milk and bananas

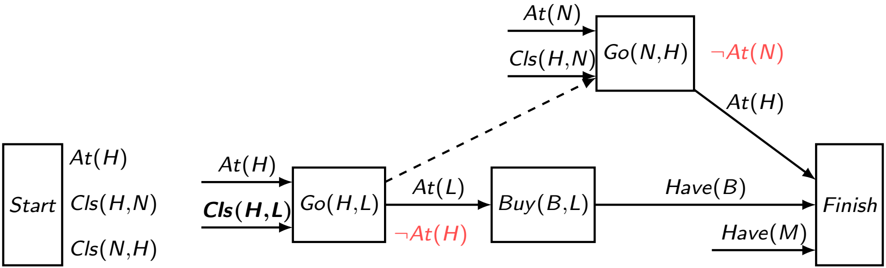


POP example: Milk and bananas



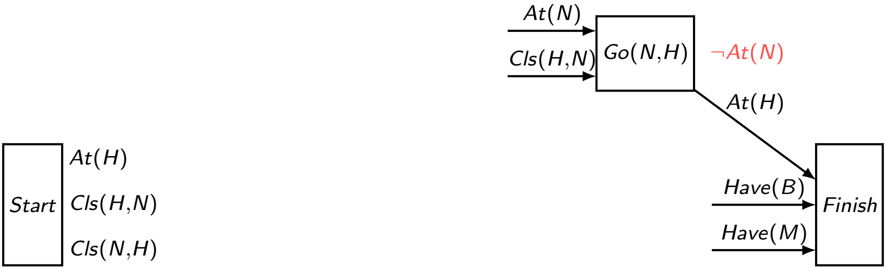
Action $Go(H, L)$ conflicts with ordering constraint $Go(N, H) \overset{At(H)}{\prec} Finish$, so we add the ordering constraint $Go(H, L) \prec Go(N, H)$.

POP example: Milk and bananas

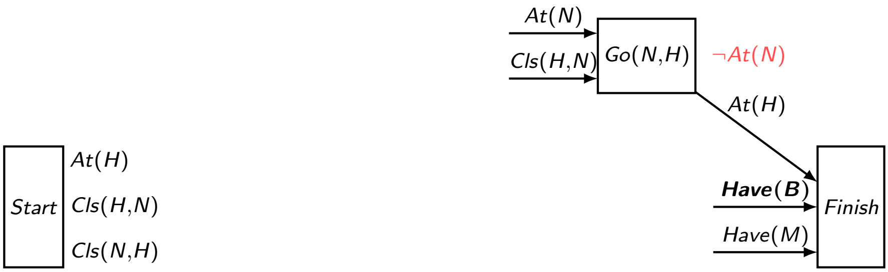


We have no have of achieving $Close(H, L)$, so we have to backtrack (but not all the way this time).

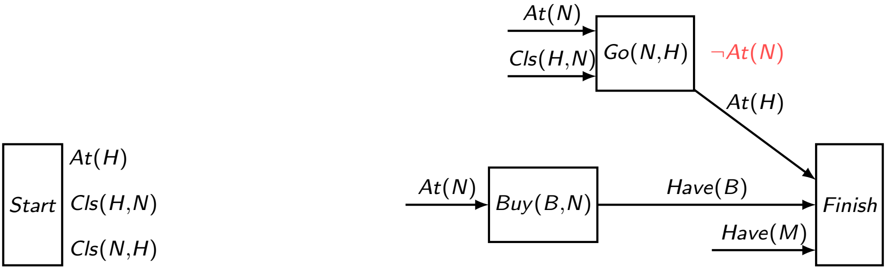
POP example: Milk and bananas



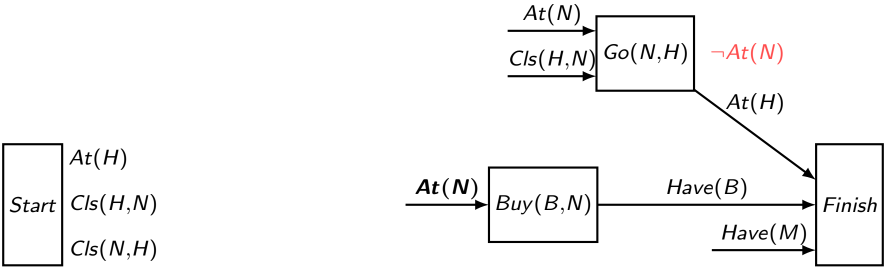
POP example: Milk and bananas



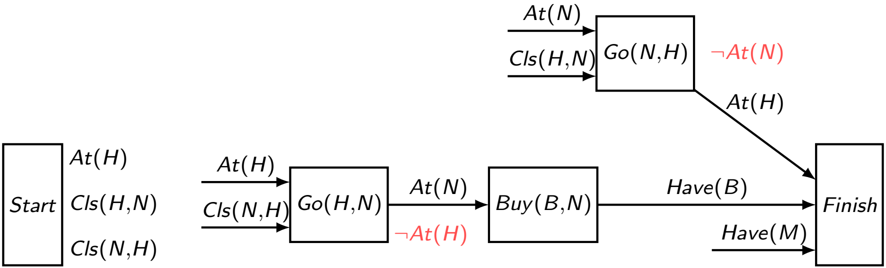
POP example: Milk and bananas



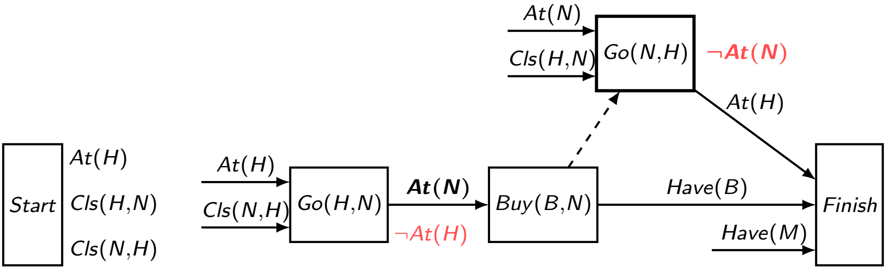
POP example: Milk and bananas



POP example: Milk and bananas

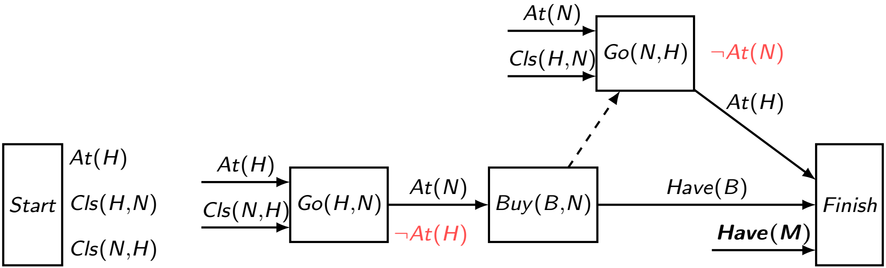


POP example: Milk and bananas

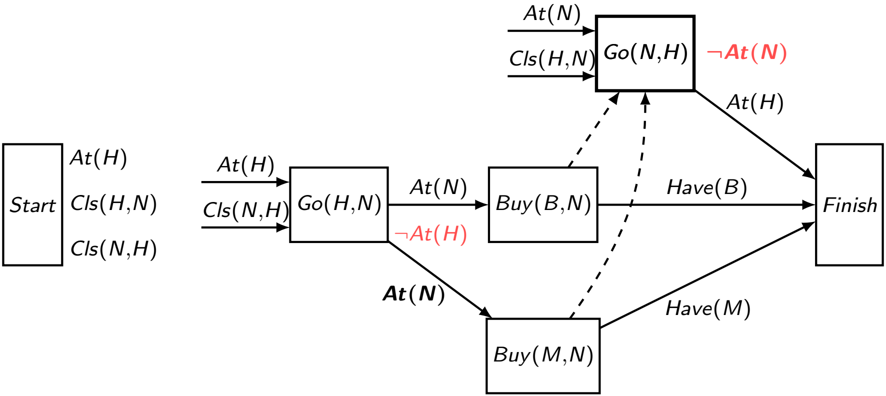


$Go(N,H)$ conflicts with ordering constraint $Go(H,N) \overset{At(N)}{\prec} Buy(B,N)$, so we add the ordering constraint $Buy(B,N) \prec Go(N,H)$.

POP example: Milk and bananas

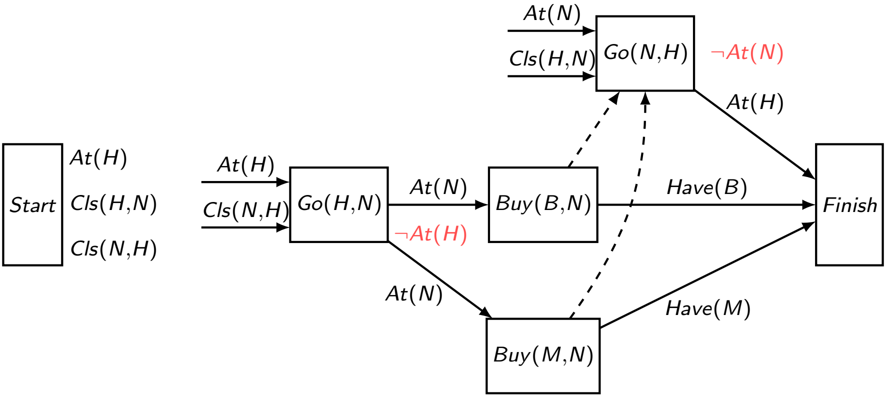


POP example: Milk and bananas

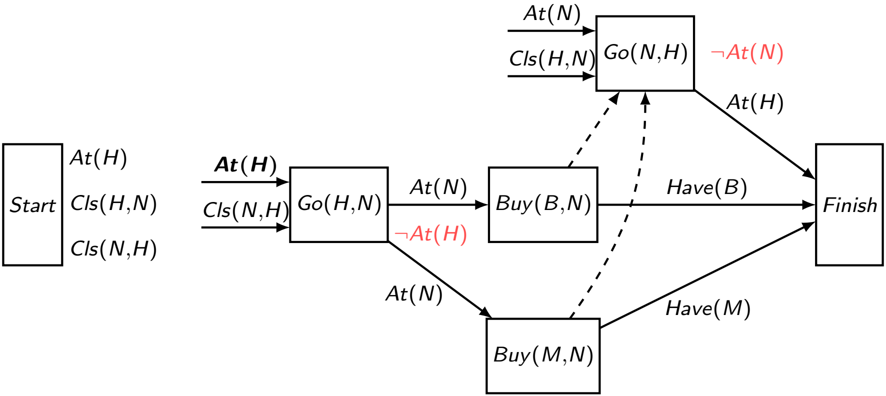


Same for buying milk: $Go(H,N) \overset{At(N)}{\prec} Buy(M,N)$.

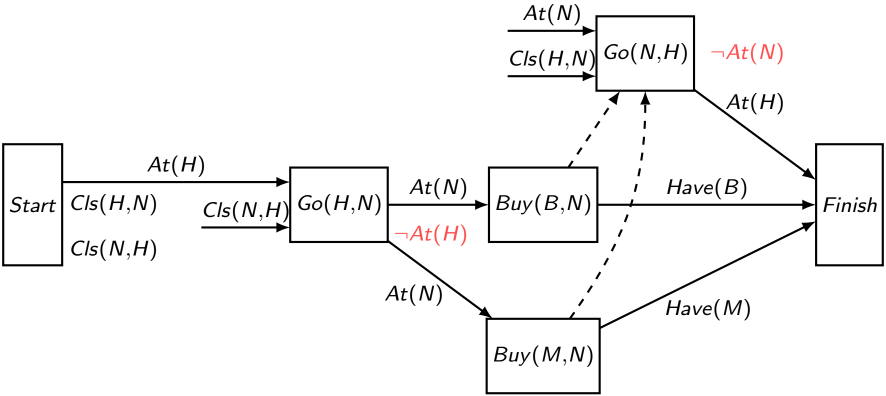
POP example: Milk and bananas



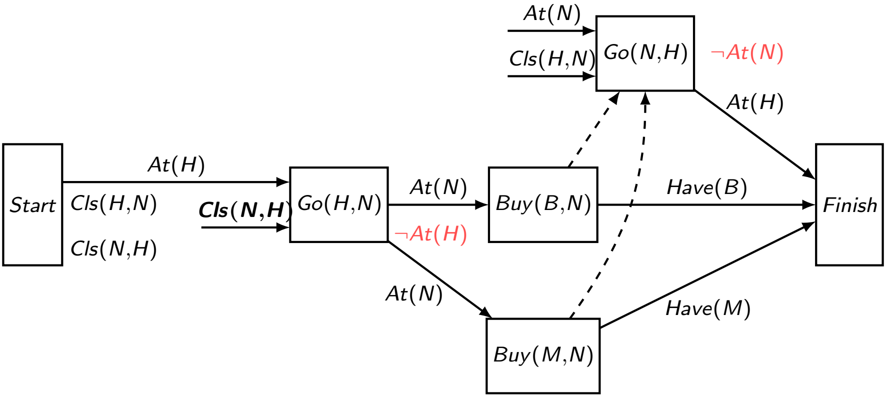
POP example: Milk and bananas



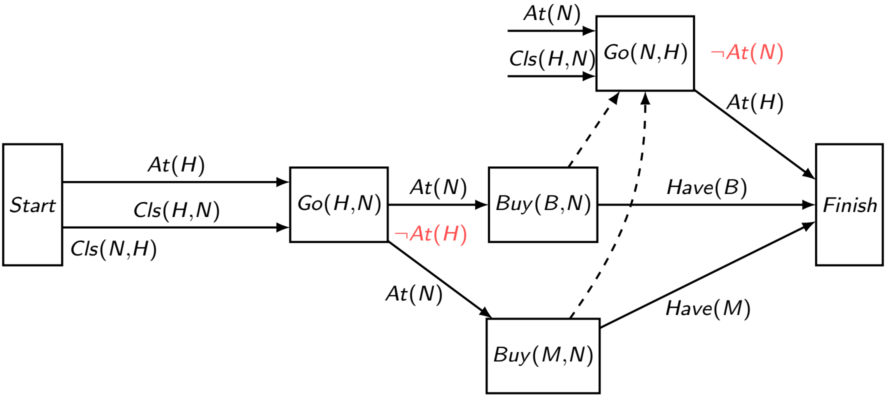
POP example: Milk and bananas



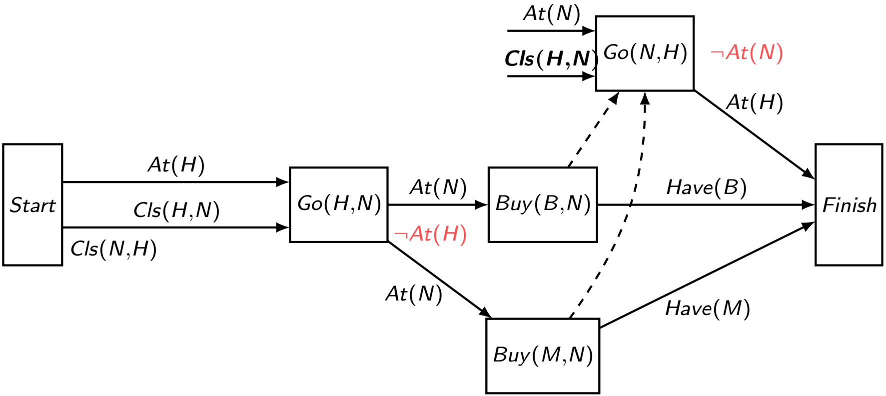
POP example: Milk and bananas



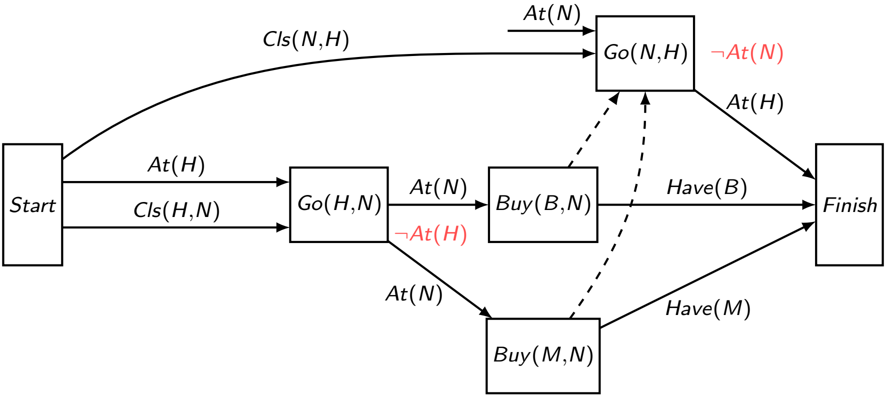
POP example: Milk and bananas



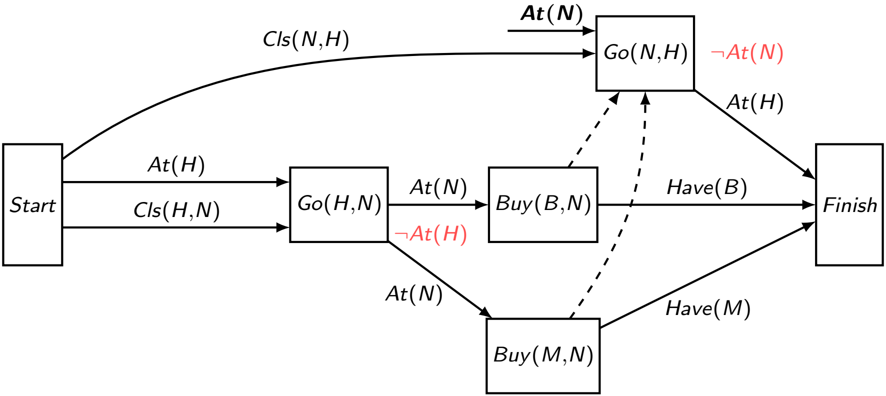
POP example: Milk and bananas



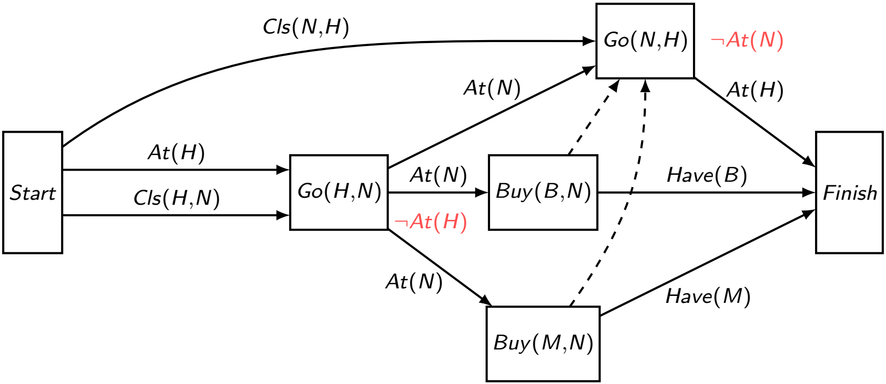
POP example: Milk and bananas



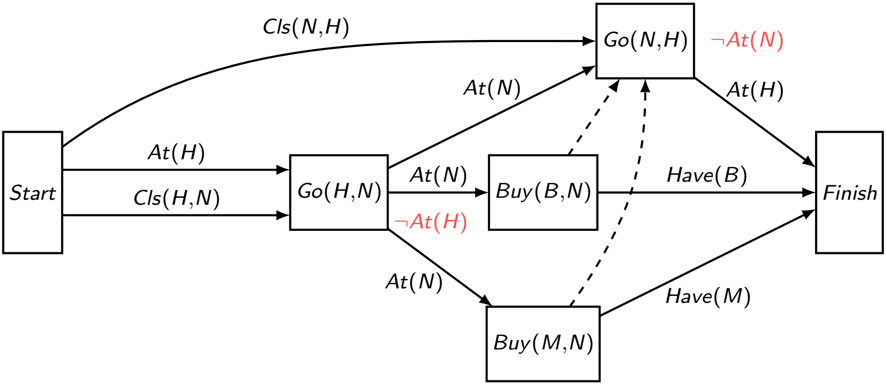
POP example: Milk and bananas



POP example: Milk and bananas



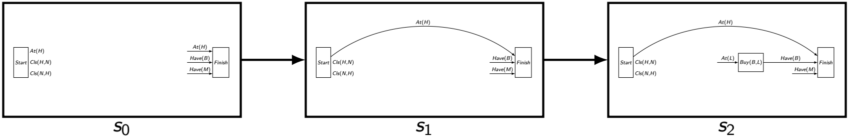
POP example: Milk and bananas



The linearisations are $Go(H, N)$, $Buy(B, N)$, $Buy(M, N)$, $Go(N, H)$ and $Go(H, N)$, $Buy(M, N)$, $Buy(B, N)$, $Go(N, H)$.

POP algorithm

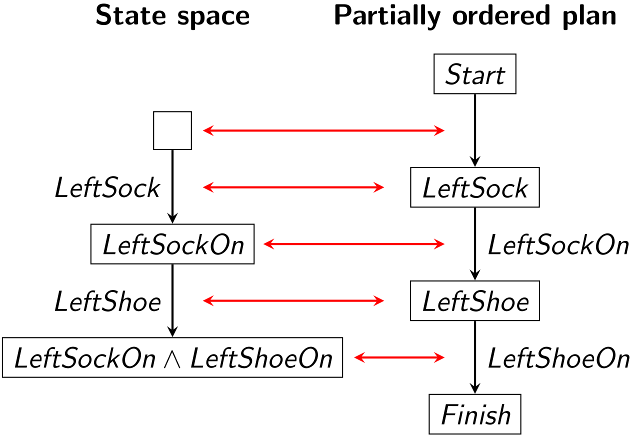
- The POP algorithm can be seen as solving a search problem in the state space consisting of partial POP graphs. The initial state in this state space is the POP graph only containing the Start and Finish nodes. We search this state space in a depth-first manner, backtracking when reaching a POP graph having a conflict that can't be resolved. Using heuristics, we can use informed search instead (finding the best open preconditions and most promising actions to achieve them first).
- When running the POP algorithm by hand, take advantage of the fact that you have a good heuristic intuition for these small examples (e.g., you probably wouldn't need to try Lidl before Netto).



Duality

Note that partially ordered plans are **dual** to state space graphs! (Nodes in one is edges in the other.)

Example. Initial state is empty, goal is *LeftShoeOn*.



POP Summary

Properties of the **POP algorithm**:

- Works from goal to start state (as regression planning and the additive/max heuristics).
- Builds a graph (partial order plan) dual to a state space graph.
- Any linearisation of the partial order will constitute a solution to the planning problem.
- **Advantages**:
  - Particularly useful for partly decomposable problems. Why? Because one doesn't have to decide on the ordering of independent actions.
  - Good for **replanning** in dynamic worlds.
  - Extends naturally to **planning and scheduling problems**.
  - Generates human-friendly plans.
  - Important for applications (e.g. Mars rovers, Maersk container shipping).
- **Disadvantage**: Doesn't currently scale up as well to large problems as state-space planners with good heuristics (hard to come up with good heuristics for POP).

Hierarchical planning

Partial-order planning explores **problem decomposition** to gain efficiency: Independent subgoals are worked on independently.

An alternative problem decomposition is **hierarchical decomposition**: Split **complex tasks** into a small number of **subtasks** that each in turn consists of a small number of subtasks, etc.

Humans have a similar approach to planning.

Example. Assume my goal is to have lunch.

- Top-level plan: *FindFood*, *EatFood*.
- *FindFood* can be split into *ChooseShop*, *BuyFoodAtShop*.
- *BuyFoodAtShop* can e.g. be split into *GoToShop*, *BuyFood*, *GoHome*.

Contrast this with basic planning at the atomic level of, say, individual body movements.

HTN planning

Hierarchical planning can reduce the combinatorial explosion resulting from planning at the atomic level.

Hierarchical Task Network planning (**HTN planning**): A planning method exploring the ideas of hierarchical planning.

The **essential component** of HTN planning is **action decompositions**.

Action decompositions (or **refinements**): The reduction of a **high-level action** (**HLA**) to a sequence of lower-level actions.

HTN planning algorithm: HLA's are recursively refined until only **primitive actions** (non-refinable actions) remain.

Implementation of an HLA: The result of a recursive refinement into primitive actions. Usually not unique.

HTN refinements

HTN **refinements** are represented by **refinement schemas**.

Definition (Refinement schema)

A **refinement schema** consists of the following elements:

- **HLA name**. The name of the HLA that the refinement schema defines a decomposition for. E.g. $GetFood$, $Buy2Things(x, y)$ or $Navigate(x, y)$.
- **Precondition**. A conjunction of literals. E.g. $At(Shop) \wedge Sells(Shop, x) \wedge Sells(Shop, y)$. Tells which conditions have to be satisfied in order for the refinement to be applicable.
- **Steps**. A sequence of actions (a totally ordered plan). These actions can either be primitive or high-level. E.g. $[Buy(x, Shop), Buy(y, Shop)]$ or $[Go(x, z, r), Navigate(z, y)]$.

Example.

REFINEMENT: $Buy2Things(x, y)$

PRECONDITION: $At(Shop) \wedge Sells(Shop, x) \wedge Sells(Shop, y)$

STEPS: $[Buy(x, Shop), Buy(y, Shop)]$

Refinement properties

- **Precondition**. Sometimes the **precondition** of a refinement schema is not explicitly stated, as it can be computed from the **steps** of the schema. How? All open preconditions when considering the steps as a partially ordered plan with no start state.
- **Multiple refinements**. There can be several refinements schemas for the same high-level action. E.g. a refinement of $Navigate(x, y)$ with steps $[Go(x, z, r), Navigate(z, y)]$ and another with just a single step $[Go(x, y, r)]$. Note the recursion!

High-level plan: A sequence of high-level and possibly primitive actions.

Achieving a goal: A high-level plan **achives a goal** if *at least one* of its implementations (recursive refinements) does.

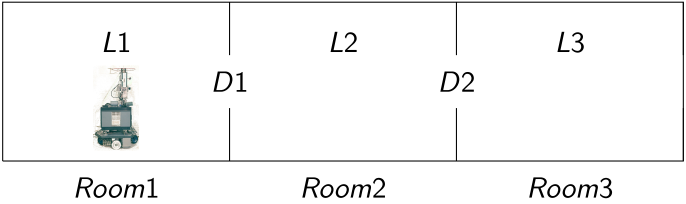
HTN planning example

Example. Refinement schemas:

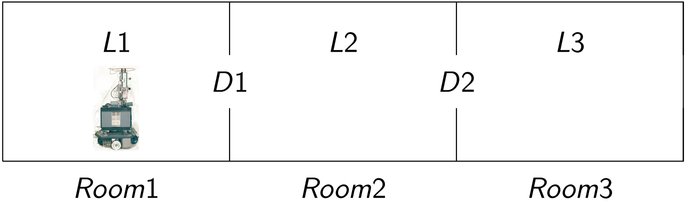
REFINEMENT: *Navigate*(x, y)
PRECONDITION: $At(x) \wedge In(x, r) \wedge In(z, r)$
STEPS: $[Go(x, z, r), Navigate(z, y)]$

REFINEMENT: *Navigate*(x, y)
PRECONDITION: $At(x) \wedge In(x, r) \wedge In(y, r)$
STEPS: $[Go(x, y, r)]$

Initial state:



Assume Shakey is asked to carry out the HLA *Navigate*$(L1, L3)$....



Implementation of the HLA $Navigate(L1, L3)$:

1. Refine $Navigate(L1, L3)$ into
$$[Go(L1, D1, Room1), Navigate(D1, L3)].$$
2. Then further into:
$$[Go(L1, D1, Room1), Go(D1, D2, Room2), Navigate(D2, L3).$$
3. And finally into:
$$[Go(L1, D1, Room1), Go(D1, D2, Room2), Go(D2, L3, Room3)].$$

$Navigate(L1, L3)$ **achieves the goal** $At(L3)$.

Representing high-level actions

To allow planning in terms of HLA's before refinement, HLA's are specified using PDDL/STRIPS **action schemas**.

Example. The $Navigate(x, y)$ HLA can be represented by the following PDDL/STRIPS **action schema**:

ACTION: $Navigate(x, y)$
PRECONDITION: $At(x)$
EFFECT: $At(y) \wedge \neg At(x)$

Definition (Downward refinement property)

An HLA action schema satisfies the **downward refinement property** if at least one implementation of the HLA achieves the stated EFFECT of the schema from the stated PRECONDITION.

Example. The HLA action schema above has the downward refinement property wrt. the refinements defined earlier.

Note: We are here presenting a simplified version of HTN where the effects of a high-level action are unique, not using **angelic semantics** as in R&N 3ed.

HTN planning example

Example. Assume now that the robot can only navigate through a room if the light is on in the room:

ACTION: $Go(x, y, r)$
PRECONDITION: $At(x) \wedge In(x, r) \wedge In(y, r) \wedge Light(r)$
EFFECT: $At(y) \wedge \neg At(x)$

This modification causes a change in the preconditions of the $Navigate(x, y)$-refinements:

REFINEMENT: $Navigate(x, y)$
PRECONDITION: $At(x) \wedge In(x, r) \wedge In(z, r) \wedge Light(r)$
STEPS: $[Go(x, z, r), Navigate(z, y)]$

REFINEMENT: $Navigate(x, y)$
PRECONDITION: $At(x) \wedge In(x, r) \wedge In(y, r) \wedge Light(r)$
STEPS: $[Go(x, y, r)]$

HTN planning example (cont'd)

Recall the action schema of $Navigate(x, y)$:

ACTION: $Navigate(x, y)$
PRECONDITION: $At(x)$
EFFECT: $At(y) \land \neg At(x)$

With the refinement modifications carried out above, $Navigate(x, y)$ unfortunately no longer satisfies the downward refinement property. Why? If the light is off, no navigation is possible.

Naive solution: Add to the precondition of the action schema of $Navigate(x, y)$ that the light should be on in all rooms. This is no good. Why? Impractical and only works if the domain (number and name of rooms) is fixed.

Can you think of a better solution? Redefine refinements to allow turning on the light. See following slides.

HTN planning example (cont'd)

Better solution: Replace $Go(x, y, r)$ in the steps of the $Navigate(x, y)$-refinements by the following HLA $Go'(x, y, r)$:

ACTION: $Go'(x, y, r)$
PRECONDITION: $At(x) \wedge In(x, r) \wedge In(y, r)$
EFFECT: $At(y) \wedge \neg At(x) \wedge Light(r)$

REFINEMENT: $Go'(x, y, r)$
PRECONDITION:
$At(x) \wedge In(x, r) \wedge In(y, r) \wedge Light(r)$
STEPS: $[Go(x, y, r)]$

REFINEMENT: $Go'(x, y, r)$
PRECONDITION:
$At(x) \wedge In(x, r) \wedge In(y, r)$
STEPS: $[TurnOn(x, r), Go(x, y, r)]$

Here $TurnOn(x, r)$ is a **primitive action** for turning on the light in the current room:

ACTION: $TurnOn(x, r)$
PRECONDITION: $At(x) \wedge In(x, r)$
EFFECT: $Light(r)$

Downward refinement property re-established: Light doesn't have to be on for $Navigate(x, y)$ to be executable.

HTN planning example (cont'd)

We now have the following 3-level action hierarchy (primitive actions not shown):

REFINEMENT: $Navigate(x, y)$
PRECONDITION: $At(x) \wedge In(x, r) \wedge In(z, r)$
STEPS: $[Go'(x, z, r), Navigate(z, y)]$

REFINEMENT: $Navigate(x, y)$
PRECONDITION: $At(x) \wedge In(x, r) \wedge In(y, r)$
STEPS: $[Go'(x, y, r)]$

REFINEMENT: $Go'(x, y, r)$
PRECONDITION:
$At(x) \wedge In(x, r) \wedge In(y, r) \wedge Light(r)$
STEPS: $[Go(x, y, r)]$

REFINEMENT: $Go'(x, y, r)$
PRECONDITION: $At(x) \wedge In(x, r) \wedge In(y, r)$
STEPS: $[TurnOn(x, r), Go(x, y, r)]$

ACTION: $Navigate(x, y)$
PRECONDITION: $At(x)$
EFFECT: $At(y) \wedge \neg At(x)$

ACTION: $Go'(x, y, r)$
PRECONDITION:
$At(x) \wedge In(x, r) \wedge In(y, r)$
EFFECT:
$At(y) \wedge \neg At(x) \wedge Light(r)$

Note the increase in **planning efficiency**: when planning the route, no branching is produced from considering whether the light is on or not. This problem is dealt with *after* the overall route has been planed.

HTN planning example (cont'd)

We can add further levels on top of our existing plan hierarchy:

ACTION: $MoveBox(b, x, y)$
PRECONDITION: $On(b, x)$
EFFECT: $On(b, y) \wedge \neg On(b, x)$

REFINEMENT: $MoveBox(b, x, y)$
PRECONDITION: $At(l) \wedge On(b, x)$
STEPS: $[Navigate(l, x), NavWithBox(b, x, y), Navigate(y, l)]$

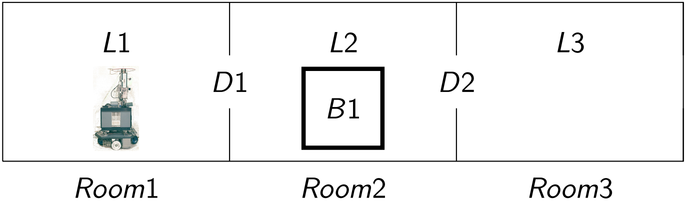
ACTION: $NavWithBox(b, x, y)$
PRECONDITION: $At(x) \wedge On(b, x)$
EFFECT: $At(y) \wedge \neg At(x) \wedge On(b, y) \wedge \neg On(b, x)$

REFINEMENT: $NavWithBox(b, x, y)$
PRECONDITION: $At(x) \wedge On(b, x) \wedge In(x, r) \wedge In(z, r)$
STEPS: $[Push'(b, x, z, r), NavWithBox(b, z, y)]$

REFINEMENT: $NavWithBox(b, x, y)$
PRECONDITION: $At(x) \wedge On(b, x) \wedge In(x, r) \wedge In(y, r)$
STEPS: $[Push'(b, x, y, r)]$

HTN planning example (cont'd)

We now have an HLA $MoveBox(b, x, y)$ for moving arbitrary boxes between arbitrary locations. Consider the following scenario:



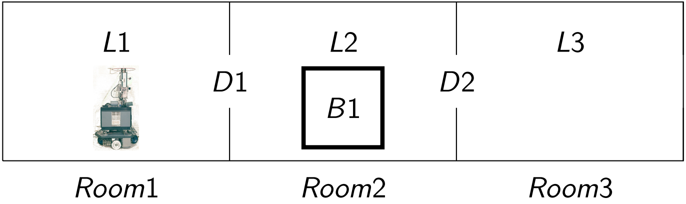
Assume our **goal** is $On(B1, L3)$.

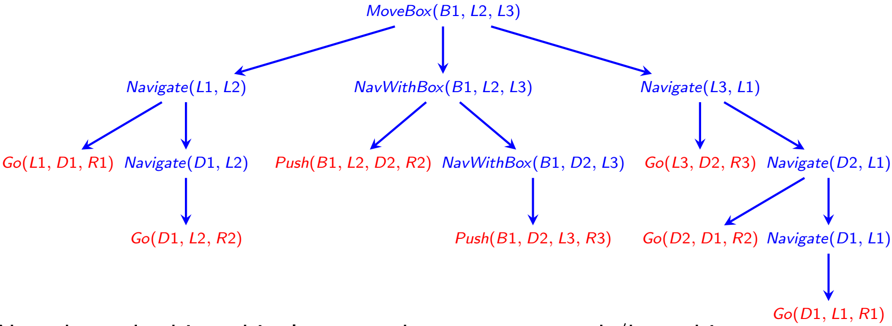
The HLA $MoveBox(B1, L2, L3)$ **achieves** this goal.

The following slide shows a possible implementation.

Remark. For simplicity, we ignore the light aspect (thus using *Go* instead of Go' and *Push* instead of $Push'$).

Implementation example





Note how the hierachical approach saves on search/branching.

HTN planning advantages

Huge advantage of HTN planning: Can be arbitrarily adjusted along the axis from **purely imperative** (no search) to **purely declarative** (only search):

HTN

Purely imperative.
Single top level action.
Unique implementation.
Linear time.

Purely declarative.
Only primitive actions.
Exponential time.

Allows for combinations of subtasks of different types:

- Trivial ones with unique (or few) implementations. (Solution is given).
- Subtasks corresponding to classical planning problems. (Solution completely unknown).

HTN planning advantages

Advantages of HTN planning:

- Allowing **hierarchical decomposition**, an approach widely used in "real life" (consider e.g. writing a large report or planning your vacation).
- "Help" the planner by telling how complex tasks can be decomposed. Can be the crucial difference between **intractability** and **tractability** for **large-scale applications**.

HTN planning has been more popular in **applications** and **industry** than any other planning method:

- Used for production-line scheduling, spacecraft planning and scheduling, equipment configuration, manufacturing process planning, evacuation planning, bridge computers, robotics.
- Used by companies such as Hitachi, Price Waterhouse, Jaguar Cars, and the Danish computer game company IO Interactive.